

A scenic mountain landscape with green hills, rocky cliffs, and a cloudy sky. The foreground shows a rocky, grassy slope. In the middle ground, there are steep, rocky cliffs and green hills. The background features more distant, hazy mountain ranges under a sky filled with white and grey clouds.

AI & Software Engineering Workshop

Week 1, Day 3: New Commands

Edik Simonian, Summer 2026

Decorators in 60 seconds

```
@bot.message_handler(commands=["start"], func=is_allowed)
def cmd_start(message):
    ...
```

- A decorator **registers** your function with the bot
- *"When a message matches `/start`, call this function"*
- You don't call `cmd_start` yourself; the bot does
- That's all you need to know to add any command

The built-ins: read before you write

Command	What it does
<code>/start</code>	Welcome message (you already edited it)
<code>/help</code>	Lists commands (you edited it yesterday)
<code>/reset</code>	Clears your conversation history
<code>/about</code>	Model, storage, and hosting info

`/reset` clears *history*. Wait, what history? **That's tomorrow.**

Live-code: `/joke`

```
@bot.message_handler(commands=["joke"], func=is_allowed)
def cmd_joke(message):
    bot.send_message(
        message.chat.id,
        "Why do programmers prefer dark mode? Because light attracts bugs!",
    )
```

1. Add the handler to `bot/handlers.py`
2. Add `/joke` to `/help`
3. Restart, test

Your turn: pick one

- `/quote`: a random quote from a list you write
- `/fact`: a surprising fact (keep a few in a Python list)
- `/roll`: a dice roll (`random.randint` is your friend)
- `/flip`: heads or tails

Same recipe as `/joke`: handler → `/help` entry → restart → test.

Build the first by hand; pair with Claude Code on the next, but you explain it in the review circle.

Level up: commands that think

`/roast <name>` : read the words after the command, send them to the AI:

```
@bot.message_handler(commands=["roast"], func=is_allowed)
def cmd_roast(message):
    name = message.text.split(maxsplit=1)[1] if " " in message.text else "you"
    reply = ask_ai(message.from_user.id, f"Write a short, playful, friendly roast of {name}.")
    bot.send_message(message.chat.id, reply)
```

`message.text.split()` : that's argument parsing.

(The repo's `/model` handler does the same trick; it activates in Week 2.)

Lock it in with a test

- Open `tests/test_handlers.py`: see how `/start` is tested
- Write the same shape of test for **your** command
- `make test`: green locally
- `git push`: green on GitHub Actions too

A command without a test is a command that breaks silently later.

Code review circle

1. Rotate one seat to your left, to your neighbor's screen
2. Read the handler on that screen **aloud**, line by line
3. Explain what each line does; ask if you can't
4. One compliment, one suggestion, then rotate back

Reading other people's code is half of real software engineering.

Today → Tomorrow

Today your bot has custom commands: some static, one that thinks.

Tomorrow: memory. The bot remembers conversations across restarts, and you'll find out what `/reset` actually resets.